

開源軟體繁體中文在地化翻譯工具：`replace.sh`

課程連結

這份報告是我將自己在課外開發的翻譯工具，結合「專題閱讀與研究」課程要求所整理出的完整專題。

一、研究動機

在長期參與開源社群的過程中，我發現大量專案的繁體中文翻譯是從簡體直接轉換而來，充斥著不符台灣習慣的詞彙（如軟體被寫成軟件、記憶體被寫成內存），甚至有些專案全無繁體中文版本。起初我手動逐行修正並提交 `pull request`，但同樣的錯誤在不同專案反覆出現，手動處理根本無法應對數十個專案的規模。因此我開發了自動化工具 `replace.sh` 來解決這個重複性的問題。

二、技術架構與功能

核心功能

- 語言：`Bash`（約 1800 行）
- 檔案搜尋：`fd + ripgrep`（高速檔案發現與內容匹配）
- 轉換核心：`OpenCC` 簡繁轉換引擎 + 自訂 `sed` 字典檔輔助（`dict.txt`）
- 支援格式：純文字、`json`、`png(metadata)` 等
- 支援方向：`CN→TW`（預設） / `TW→CN`（反向模式）

額外功能

- 檔名同步翻譯：處理檔案內容的同時自動轉換檔名中的簡體字
- 並行處理模式（`-p`）：面對大量檔案時分散運算提升效率
- `git` 整合：讀取 `.gitignore`（`-g`），自動加入版本控制暫存區（`--stage`）
- 安全機制：備份（`-b`）、`dry-run` 預覽（`-n`）、日誌輸出（`-l`）、覆寫警告

歧義詞處理

部分詞彙在繁體中文有多重含義，不能無條件替換。例如「質量」同時表示品質與物理學上的質量，若一律轉換會造成科學文本的謬誤。因此我將替換規則分為主字典（**dict.txt**）與爭議性字典（**controversial.txt**）兩層，後者需要使用者以 **-c** 參數主動啟用，避免誤判。

三、開發歷程與遭遇的困難

初期：單純的字串替換

最早版本只是一支簡單的 **sed** 替換腳本，處理明確對應的詞彙。不過初期的腳本問題連連。程式碼區塊內的特定英文字串被意外修改，或者部分單字被錯誤轉換都是之後需要解決的。

中期：引入分層替換與白名單

為了解決誤判，我在架構中引入 **OpenCC** 作為基礎轉換引擎，再用自訂的 **sed** 字典檔處理 **OpenCC** 無法涵蓋的臺灣特用詞。同時將歧義詞獨立成 **controversial** 字典，由使用者自行決定啟用與否。開發過程中我使用 **AI** 工具輔助實作各功能模組的程式碼，自己則專注在功能規劃、歧義詞字典的維護與輸出品質的把關。

後期：效能優化與格式擴展

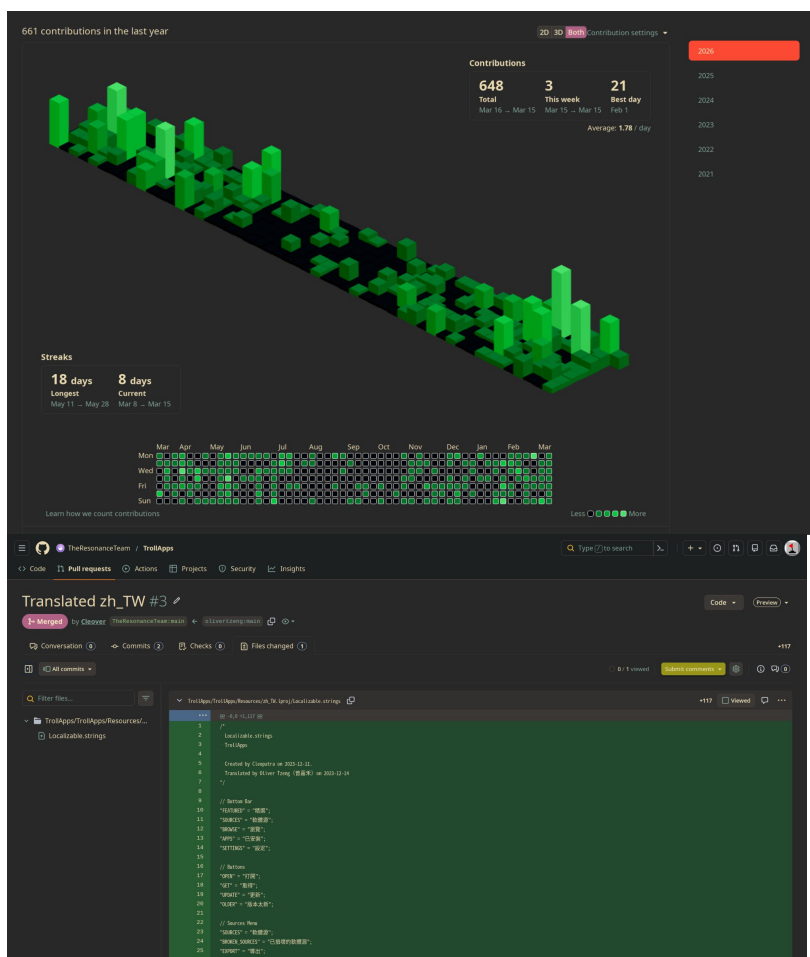
專案數量增加後，逐檔處理的速度成為瓶頸。架構上改用 **fd** 與 **ripgrep** 取代原本的 **find** 與 **grep**，大幅提升檔案搜尋與內容匹配的速度，並加入並行處理模式應對大型專案。同時擴展支援格式至 **JSON value** 的選擇性翻譯。

持續維護

工具的字典檔存放於我的 **dotfiles** 中，根據自己在各專案提交 **PR** 時遇到的新案例持續更新。每一次新的誤判都會回饋到字典中，讓工具的覆蓋率逐步提升。

四、成果

- 累計 145 個 pull request 被上游專案合併
- 涵蓋專案包括：topgrade-rs、yay、BleachBit 等開源工具
- 工具本身開源於 GitHub，供其他貢獻者使用



五、反思

從最初的簡單替換腳本到目前約 1800 行的工具，最大的收穫是學會把「想偷懶」變成一個真正能解決問題的系統。一開始只是覺得手動翻譯很麻煩，後來慢慢拆解出哪些能自動修、哪些不能動。開發過程中雖然大量依賴 AI 工具輔助產出程式碼，但功能的規劃、歧義詞的判斷與每一筆翻譯的品質把關始終是我自己的工作。replace.sh 的經驗讓我養成了一個習慣：當我發現自己在不同專案反覆做同樣的事，就開始想辦法把它自動化。後續我

開發 **yt.py** 自動化音樂庫管理，並以 **Docker** 部署十多項自架服務，皆是這套邏輯的延伸。這個從抱怨、拆解到實作的過程，是我高中階段最扎實的一課。

附錄

```
Advanced Chinese Converter & PNG Metadata Tool
Uses fd + rg for fast file discovery and content matching.

Usage: replace.sh [OPTIONS] [files...]

Conversion Options:
  -c      Enable controversial replacements
  -r      Reverse mode: TW -> CN (default: CN -> TW)
  -i      In-place mode: overwrite original files
           Also translates filenames (刪除.txt -> 刪除.txt)
  -R      Recursive: process directories recursively
           (auto-enabled when input is a directory)
  -u      Unify: force -TW/-CN suffix on output
  --new-api Translate JSON "translation" values only (keys untouched)

PNG Operations:
  -j [MODE] PNG extraction: o(riginal), c(n), t(w)
           cn/tw modes also translate filename
  -a      Amend mode: combine PNG + JSON into character card
           Usage: -a <file1> <file2> [-o output.png]

File Handling:
  -H      Include hidden files
  -g      Respect .gitignore (default: on)
  -b      Create backup (.bak)
  -o [files] Specify output filenames
  -s, --stage Auto git-stage processed files

Execution:
  -n      Dry-run (no changes made)
  -p      Parallel processing (faster for many files)
  -f      Force continue on errors
  -q, --quiet Quiet mode (errors only)
  -l, --log FILE Write log to file
  --no-confirm Skip confirmation prompt for -i -R

Help:
  --help  Show this help

Examples:
  replace.sh -i file.txt           In-place convert (CN->TW)
  replace.sh -i -r file.txt       In-place convert (TW->CN)
  replace.sh ./zh-CN/            Copy to ./zh-TW/ with converted files
  replace.sh -i ./zh-CN/         In-place convert with dir rename
  replace.sh -i -R -p ./dir/      Parallel recursive conversion
  replace.sh -i -R -b -s ./dir/   With backup and auto-stage
  replace.sh -a image.png data.json Combine into character card
  replace.sh --new-api -i file.json Translate translation values only

Supported Extensions:
  txt json jsonl md po strings png PNG yaml yml xyaml js mjs html css scss

Blacklist:
  *.bak *.tmp .DS_Store .git LICENSE __pycache__ build dist node_modules
```

- **replace.sh** GitHub 連結：
<https://github.com/olivertzeng/dotfiles/blob/main/replace.sh>
- **yt.py** GitHub 連結：<https://github.com/olivertzeng/dotfiles/blob/main/yt.py>